COMPLETE MASTER GUIDE

PHP

Server-Side Web Development from Zero to Confident

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

WHAT THIS COMPLETE GUIDE COVERS

TABLE OF CONTENTS

Chapter 1 — What Is PHP?

PHP — which stands for PHP: Hypertext Preprocessor (a recursive acronym) — is the most widely used server-side scripting language on the internet. While HTML, CSS, and JavaScript run in the user's browser, PHP runs on the web server before any content is sent to the browser. This makes it the engine behind dynamic websites, login systems, databases, and web applications.

According to W3Techs, PHP powers over 77% of all websites on the internet — including giants like Facebook (originally), WordPress, Wikipedia, and millions of Kenyan business and government platforms.

1.1 A Brief History of PHP

Year	Milestone
1994	Rasmus Lerdorf creates PHP as 'Personal Home Page Tools' — simple scripts to track visits to his CV.
1997	Zeev Suraski and Andi Gutmans rewrite PHP's core. PHP 3 is released — a real programming language.
2000	PHP 4 — the Zend Engine brings massive performance improvements.
2004	PHP 5 — true OOP support with classes, interfaces, and exceptions.
2015	PHP 7 — nearly 2x faster than PHP 5. Type declarations, null coalescing, spaceship operator.
2020	PHP 8 — JIT compiler, named arguments, match expression, nullsafe operator.
Today	PHP 8.3+ is the current version — actively maintained, modern, and fast.

1.2 Why Learn PHP?

Reason	Why It Matters
Powers WordPress	65% of all CMS-powered websites use WordPress — built entirely in PHP.
Backend of the web	Login systems, contact forms, user dashboards, e-commerce — all PHP.
Works with MySQL	PHP + MySQL is the most common web development stack globally.
Beginner-friendly	Easy to learn, runs on free tools (XAMPP/WAMP), huge community.
Job market	PHP developers are in demand across Kenya, Africa, and worldwide.
Full ecosystem	Frameworks: Laravel, Symfony, CodeIgniter. CMS: WordPress, Drupal, Joomla.

1.3 How PHP Works — The Server-Side Execution Model

1. User's browser sends a request: https://mysite.co.ke/profile.php
2. The web server (Apache/Nginx) receives the request and sees a .php file.
3. The server passes the file to the PHP interpreter (engine).
4. PHP reads the file from top to bottom, executes all PHP code.
5. PHP generates pure HTML output as its result.
6. The server sends that HTML back to the browser.
7. The browser displays the page — it never sees any PHP code.

KEY INSIGHT — PHP vs JavaScript
JavaScript runs in the BROWSER (client-side). Users can see JS code in DevTools.
PHP runs on the SERVER. Users NEVER see PHP code — only the HTML it produces.
This is why PHP is trusted for passwords, database queries, and business logic.
PHP code is completely hidden from the end user.

1.4 Setting Up a PHP Development Environment

Tool	Purpose & Notes
XAMPP	Free package for Windows/Mac/Linux. Includes Apache, PHP, MySQL, and phpMyAdmin. Most popular choice for beginners.
WAMP	Windows only. Apache + MySQL + PHP in one installer.
MAMP	Mac only. Apache + MySQL + PHP.
Laragon	Windows. Lightweight, fast, and beginner-friendly. Excellent alternative to XAMPP.
VS Code	Free code editor with PHP IntelliSense extension. Recommended.
phpMyAdmin	Browser-based GUI for managing MySQL databases. Included with XAMPP.

1.5 Your First PHP File

```
<?php
// This is a PHP comment

// Output text to the browser
echo "Hello, World!";

// PHP inside HTML — the most common pattern
?>

<!DOCTYPE html>
<html lang="en">
<head>
  <title>My First PHP Page</title>
</head>
<body>
  <h1>
    <?php echo "Welcome to PHP!"; ?>
  </h1>
  <p>The current date is: <?php echo date('d F Y'); ?></p>
</body>
</html>
```

HOW TO RUN PHP FILES

1. Install XAMPP — download from apachefriends.org
2. Start Apache and MySQL from the XAMPP Control Panel.
3. Save your .php files in: C:\xampp\htdocs\ (Windows) or /opt/lampp/htdocs/ (Linux)
4. Open your browser and go to: http://localhost/yourfile.php
5. NEVER open .php files by double-clicking — they must be served through Apache.

Chapter 2 — PHP Syntax, Variables & Data Types

2.1 PHP Syntax Rules

- All PHP code must be inside <?php ... ?> tags.
- Every statement ends with a semicolon ;
- PHP is case-insensitive for keywords (echo, ECHO, Echo all work) but case-sensitive for variable names.
- Comments: // single-line, /* multi-line */, # also single-line.
- Files must be saved with the .php extension and served through a web server.

2.2 Variables

In PHP, every variable starts with a dollar sign ($). Variables are dynamically typed — you do not declare the type; PHP figures it out from the value you assign.

```
<?php
// Variable names start with $ followed by a letter or underscore
$name    = 'Lucky Nakola';      // String
$age     = 30;                  // Integer
$salary  = 75000.50;            // Float
$isActive = true;               // Boolean
$address  = null;               // Null

// Variable names are case-sensitive
$City = 'Nyeri';
$city = 'Nairobi';
// $City and $city are TWO different variables

// Output variables
echo $name;                          // Lucky Nakola
echo "Hello, $name!";               // Hello, Lucky Nakola!  (double quotes parse
vars)
echo 'Hello, $name!';               // Hello, $name!         (single quotes do
NOT)
echo "Name: {$name}, Age: {$age}";  // Best practice: wrap in {}

// Check if a variable is set
var_dump($age);        // int(30)
var_dump($name);       // string(12) "Lucky Nakola"
isset($name);          // true
isset($unknown);       // false
unset($age);           // Destroy a variable
?>
```

2.3 PHP Data Types

Type	Description	Example
String	Text — any sequence of characters	$name = 'Lucky';
Integer	Whole numbers (positive or negative)	$score = 95;
Float	Decimal / floating-point numbers	$price = 1499.99;
Boolean	true or false (case-insensitive)	$active = true;
Array	Ordered collection of values	$langs = ['PHP','JS'];
Object	Instance of a class	$u = new User();
NULL	Variable with no value assigned	$data = null;
Resource	Reference to external resource (DB conn, file handle)	$conn = mysqli_connect(...)

2.4 Constants

```php
<?php
// Constants — values that NEVER change
// define() — works everywhere (global scope)
define('SITE_NAME', 'Get Techy With Lucky');
define('MAX_LOGIN_ATTEMPTS', 5);
define('PI', 3.14159);

echo SITE_NAME;           // Get Techy With Lucky
echo MAX_LOGIN_ATTEMPTS;  // 5

// const — used inside classes or at top level
const BASE_URL = 'https://mysite.co.ke/';

// PHP built-in constants
echo PHP_VERSION;     // e.g. 8.2.0
echo PHP_EOL;         // End-of-line character
echo PHP_INT_MAX;     // Largest integer PHP can handle
echo __FILE__;        // Full path of the current file
echo __LINE__;        // Current line number
echo __DIR__;         // Directory of the current file
?>
```

Chapter 3 — Operators, Expressions & Type Juggling

3.1 Arithmetic Operators

```
<?php
$a = 20;  $b = 6;
echo $a + $b;     // 26  — Addition
echo $a - $b;     // 14  — Subtraction
echo $a * $b;     // 120 — Multiplication
echo $a / $b;     // 3.333... — Division
echo $a % $b;     // 2   — Modulus (remainder)
echo $a ** 2;     // 400 — Exponentiation
echo intdiv(20, 6); // 3 — Integer division (PHP 7+)
?>
```

3.2 Comparison Operators

```
<?php
// == loose equality (converts types before comparing)
var_dump(5 == '5');      // true  — dangerous!
var_dump(0 == 'hello');  // true  — very dangerous!

// === strict equality (checks value AND type)
var_dump(5 === '5');     // false — correct!
var_dump(5 === 5);       // true

// <> or != loose inequality
// !== strict inequality

// Spaceship operator (PHP 7+) — returns -1, 0, or 1
echo 5 <=> 10;   // -1  (5 is less than 10)
echo 10 <=> 10;  // 0   (equal)
echo 15 <=> 10;  // 1   (15 is greater than 10)

// Null coalescing operator (PHP 7+)
$username = $_GET['user'] ?? 'Guest';
// If $_GET['user'] exists and is not null, use it; otherwise use 'Guest'
?>
```

3.3 Logical Operators

```php
<?php
$age = 22;  $hasID = true;

// AND — both must be true
if ($age >= 18 && $hasID) { echo 'Access granted.'; }

// OR — at least one must be true
if ($age >= 18 || $hasID) { echo 'Possibly allowed.'; }

// NOT — reverses the boolean
if (!$hasID) { echo 'Please show your ID.'; }

// Word forms also work (lower precedence)
if ($age >= 18 and $hasID) { echo 'OK'; }
if ($age >= 18 or $hasID)  { echo 'OK'; }
?>
```

3.4 String Operators

```php
<?php
// . (dot) concatenates strings
$first = 'Lucky';
$last  = 'Nakola';
$full  = $first . ' ' . $last;
echo $full;   // Lucky Nakola

// .= append to existing string
$message  = 'Hello';
$message .= ', World!';
echo $message;   // Hello, World!
?>
```

3.5 Type Juggling & Type Casting

```php
<?php
// PHP automatically converts types when needed (type juggling)
$sum = '10' + 5;
echo $sum;   // 15 (string '10' converted to integer 10)

// Explicit type casting
$price  = '19.99 USD';
echo (int)$price;    // 19
echo (float)$price;  // 19.99
echo (string)42;     // '42'
echo (bool)'';       // false
echo (bool)'hello';  // true
echo (array)'Lucky'; // ['Lucky']
```

```
// Type checking functions
is_int(42);       // true
is_float(3.14);   // true
is_string('hi');  // true
is_bool(true);    // true
is_null(null);    // true
is_array([]);     // true
is_numeric('42'); // true — string that contains a number
?>
```

Chapter 4 — Control Flow: Conditionals & Loops

4.1 if / elseif / else

```php
<?php
$score = 75;

if ($score >= 90) {
    echo 'Grade A — Distinction';
} elseif ($score >= 75) {
    echo 'Grade B — Credit';
} elseif ($score >= 60) {
    echo 'Grade C — Pass';
} elseif ($score >= 50) {
    echo 'Grade D — Borderline';
} else {
    echo 'Grade F — Fail';
}
// Output: Grade B — Credit

// Ternary operator
$status = ($score >= 50) ? 'Passed' : 'Failed';
echo $status;   // Passed

// Null coalescing (short ternary for null checks)
$city = $user['city'] ?? 'Unknown';
?>
```

4.2 match Expression (PHP 8)

```php
<?php
$statusCode = 404;

$message = match($statusCode) {
    200, 201 => 'Success',
    301, 302 => 'Redirect',
    400      => 'Bad Request',
    401      => 'Unauthorised',
    403      => 'Forbidden',
    404      => 'Not Found',
    500      => 'Server Error',
    default  => 'Unknown Status',
};

echo $message;   // Not Found
```

```
// match uses STRICT comparison (===)
// match throws UnhandledMatchError if no arm matches and no default
?>
```

4.3 switch Statement

```
<?php
$day = 'Wednesday';

switch ($day) {
    case 'Monday':
    case 'Tuesday':
    case 'Wednesday':
    case 'Thursday':
    case 'Friday':
        echo 'Weekday — classes running.';
        break;
    case 'Saturday':
        echo 'Weekend workshop!';
        break;
    case 'Sunday':
        echo 'Rest day.';
        break;
    default:
        echo 'Invalid day.';
}
?>
```

4.4 Loops

```
<?php
// for loop
for ($i = 1; $i <= 5; $i++) {
    echo "Lap {$i}\n";
}

// while loop
$count = 0;
while ($count < 3) {
    echo "Attempt: {$count}\n";
    $count++;
}

// do...while — always executes at least once
$num = 10;
do {
    echo "Number: {$num}\n";
    $num++;
} while ($num < 10);  // Runs once even though condition is false
```

```php
// foreach — iterate over arrays
$students = ['Alice', 'Bob', 'Carol', 'David'];
foreach ($students as $student) {
    echo "Welcome, {$student}!\n";
}

// foreach with key => value
$profile = ['name' => 'Lucky', 'city' => 'Nyeri', 'role' => 'Instructor'];
foreach ($profile as $key => $value) {
    echo "{$key}: {$value}\n";
}

// break and continue
for ($i = 0; $i < 10; $i++) {
    if ($i === 5) break;     // Stop at 5
    if ($i % 2 === 0) continue; // Skip even numbers
    echo $i . ' ';   // 1 3
}
?>
```

Chapter 5 — Functions: Built-in, User-Defined & Scope

5.1 Defining & Calling Functions

```
<?php
// Basic function
function greet($name) {
    return "Hello, {$name}! Welcome to PHP.";
}
echo greet('Lucky');   // Hello, Lucky! Welcome to PHP.

// Default parameter values
function greetUser($name = 'Guest', $greeting = 'Hello') {
    return "{$greeting}, {$name}!";
}
echo greetUser();               // Hello, Guest!
echo greetUser('Alice');        // Hello, Alice!
echo greetUser('Bob','Welcome');// Welcome, Bob!

// Type declarations (PHP 7+)
function addNumbers(int $a, int $b): int {
    return $a + $b;
}
echo addNumbers(10, 25);   // 35

// Nullable type
function findUser(int $id): ?string {
    // Returns string or null
    if ($id === 1) return 'Lucky';
    return null;
}
?>
```

5.2 Variable-Length Arguments (Variadic Functions)

```
<?php
function sum(int ...$numbers): int {
    return array_sum($numbers);
}
echo sum(1, 2, 3, 4, 5);   // 15
echo sum(10, 20);           // 30

// Spread operator to pass array as arguments
$nums = [3, 6, 9];
echo sum(...$nums);   // 18
?>
```

5.3 Scope: Global, Local & Static

```php
<?php
$globalVar = 'I am global';

function testScope() {
    // $globalVar is NOT accessible here directly
    // echo $globalVar;  // Undefined variable!

    // Use 'global' keyword to import it
    global $globalVar;
    echo $globalVar;   // I am global

    // Local variable
    $localVar = 'I am local';
    echo $localVar;
}

testScope();
// echo $localVar;  // Undefined — not accessible outside function

// Static variable — retains its value between calls
function countCalls() {
    static $count = 0;
    $count++;
    echo "Call number: {$count}\n";
}
countCalls();   // Call number: 1
countCalls();   // Call number: 2
countCalls();   // Call number: 3
?>
```

5.4 Anonymous Functions & Arrow Functions

```php
<?php
// Anonymous function (closure)
$multiply = function(int $a, int $b): int {
    return $a * $b;
};
echo $multiply(6, 7);   // 42

// Using 'use' to inherit variables from parent scope
$prefix = 'Student';
$greet  = function($name) use ($prefix) {
    return "{$prefix}: {$name}";
};
echo $greet('Alice');   // Student: Alice
```

```php
// Arrow function (PHP 7.4+) — automatically captures outer scope
$multiplier = 3;
$triple = fn($n) => $n * $multiplier;
echo $triple(10);   // 30

// Higher-order function — pass a function as argument
$students = ['Charlie', 'Alice', 'Bob'];
usort($students, fn($a, $b) => strcmp($a, $b));
print_r($students);   // ['Alice', 'Bob', 'Charlie']
?>
```

5.5 Essential Built-in PHP Functions

Function	Purpose	Example
var_dump($var)	Print type and value (debugging)	var_dump($user)
print_r($arr)	Print array/object in readable format	print_r($results)
isset($var)	true if variable exists and not null	isset($_POST['email'])
empty($var)	true if '', 0, null, false, [], '0'	empty($username)
date($format)	Return formatted current date/time	date('Y-m-d H:i:s')
time()	Unix timestamp (seconds since 1970)	time()
rand($min,$max)	Generate random integer	rand(1,100)
round($n,$p)	Round a number to decimal places	round(3.14159,2)
abs($n)	Absolute value	abs(-42)
max(...$args)	Largest value	max(3,7,1,9)
min(...$args)	Smallest value	min(3,7,1,9)
intval($val)	Convert to integer	intval('42px')
floatval($val)	Convert to float	floatval('3.14px')
number_format($n)	Format number with commas/decimals	number_format(1234567,2)
header($str)	Send HTTP header	header('Location:/login.php')
exit()	Stop script execution immediately	exit('Access denied')

Chapter 6 — Arrays: Indexed, Associative & Multidimensional

6.1 Indexed Arrays

```php
<?php
// Create
$courses = ['HTML', 'CSS', 'JavaScript', 'PHP', 'MySQL'];
$numbers = array(10, 20, 30, 40, 50);  // Old syntax — still valid

// Access by index (starts at 0)
echo $courses[0];   // HTML
echo $courses[3];   // PHP
echo $courses[count($courses)-1];   // MySQL (last element)

// Modify
$courses[2] = 'JavaScript ES6';

// Add to end
$courses[] = 'Laravel';

// Count elements
echo count($courses);   // 6
?>
```

6.2 Associative Arrays

```php
<?php
// Keys are custom strings (or integers)
$student = [
    'name'    => 'Lucky Nakola',
    'age'     => 30,
    'city'    => 'Nyeri',
    'course'  => 'PHP Development',
    'active'  => true,
];

echo $student['name'];    // Lucky Nakola
echo $student['city'];    // Nyeri

// Add / update
$student['email'] = 'lucky@example.com';
$student['age']   = 31;

// Remove a key
unset($student['active']);
```

```
// Check if key exists
if (array_key_exists('email', $student)) {
    echo 'Email: ' . $student['email'];
}

// Loop over associative array
foreach ($student as $key => $value) {
    echo "{$key}: {$value}\n";
}
?>
```

6.3 Multidimensional Arrays

```
<?php
// Array of associative arrays — the most common pattern
$students = [
    ['id'=>1, 'name'=>'Alice',  'grade'=>'A', 'score'=>92],
    ['id'=>2, 'name'=>'Bob',    'grade'=>'B', 'score'=>78],
    ['id'=>3, 'name'=>'Carol',  'grade'=>'A', 'score'=>88],
    ['id'=>4, 'name'=>'David',  'grade'=>'C', 'score'=>65],
];

// Access nested values
echo $students[0]['name'];    // Alice
echo $students[2]['score'];   // 88

// Loop and display
foreach ($students as $s) {
    echo "{$s['id']}. {$s['name']} — Grade: {$s['grade']}\n";
}
?>
```

6.4 Essential Array Functions

Function	Purpose	Example
array_push($arr,$val)	Add element(s) to end of array	array_push($list,'PHP')
array_pop($arr)	Remove and return last element	array_pop($stack)
array_shift($arr)	Remove and return first element	array_shift($queue)
array_unshift($arr,$val)	Add element(s) to beginning	array_unshift($list,'first')

Function	Purpose	Example
array_merge($a,$b)	Merge two or more arrays	array_merge($a,$b)
array_unique($arr)	Remove duplicate values	array_unique($ids)
array_keys($arr)	Return all keys as an indexed array	array_keys($student)
array_values($arr)	Return all values re-indexed from 0	array_values($filtered)
array_flip($arr)	Swap keys and values	array_flip($map)
array_search($val,$arr)	Find key of a value (returns key or false)	array_search('PHP',$list)
in_array($val,$arr)	Check if value exists (returns bool)	in_array('HTML',$courses)
array_slice($arr,$s,$l)	Extract a portion of an array	array_slice($arr,0,3)
array_splice($a,$s,$l,$r)	Remove/replace elements, returns removed	array_splice($a,1,2)
array_map($fn,$arr)	Apply callback to each element, return result	array_map('strtoupper',$names)
array_filter($arr,$fn)	Filter elements by callback	array_filter($scores,fn($s)=>$s>=50)
array_reduce($arr,$fn,$i)	Reduce array to single value	array_reduce($nums,fn($c,$v)=>$c+$v,0)
sort($arr)	Sort indexed array ascending (modifies it)	sort($scores)
rsort($arr)	Sort indexed array descending	rsort($scores)
asort($arr)	Sort assoc array by value, keep keys	asort($grades)
ksort($arr)	Sort assoc array by key	ksort($profile)

Function	Purpose	Example
usort($arr,$fn)	Sort by custom comparison function	usort($students,fn($a,$b)=>$a['score']<=>$b['score'])
count($arr)	Count number of elements	count($students)

Chapter 7 — Strings: Functions & Manipulation

Strings are everywhere in PHP — user names, email addresses, database content, HTML output. PHP has over 100 built-in string functions. These are the most important ones.

7.1 String Creation & Heredoc

```
<?php
// Single quotes — no variable parsing, fastest
$name = 'Lucky Nakola';
echo 'Hello, $name!';   // Hello, $name! (literal)

// Double quotes — variables parsed inside
echo "Hello, $name!";  // Hello, Lucky Nakola!
echo "Score: {$score}%";  // Use {} for clarity

// Heredoc — multi-line, variables parsed
$html = <<<EOT
<div class="card">
  <h2>$name</h2>
  <p>Welcome to PHP!</p>
</div>
EOT;
echo $html;

// Nowdoc — multi-line, NO variable parsing (like single quotes)
$raw = <<<'EOT'
This is $name and it will not be parsed.
EOT;
?>
```

7.2 Essential String Functions

Function	Purpose	Example
strlen($str)	Length of a string	strlen('Hello') → 5
strtolower($str)	Convert to lowercase	strtolower('HELLO') → 'hello'
strtoupper($str)	Convert to uppercase	strtoupper('hello') → 'HELLO'
ucfirst($str)	Capitalise first character	ucfirst('lucky') → 'Lucky'
ucwords($str)	Capitalise first letter of each word	ucwords('lucky nakola') → 'Lucky Nakola'

Function	Purpose	Example
trim($str)	Remove whitespace from both ends	trim(' hello ') → 'hello'
ltrim($str)	Remove whitespace from left only	ltrim(' hello')
rtrim($str)	Remove whitespace from right only	rtrim('hello ')
str_replace($s,$r,$str)	Replace all occurrences of $s with $r	str_replace('PHP','Laravel',$str)
str_ireplace()	Case-insensitive replace	str_ireplace('php','PHP',$str)
strpos($str,$find)	Find position of first occurrence	strpos('hello','l') → 2
strrpos($str,$find)	Find position of last occurrence	strrpos('hello','l') → 3
strstr($str,$find)	Return string from first match onward	strstr('user@email.com','@') → '@email.com'
substr($str,$s,$l)	Return part of a string	substr('Hello World',6,5) → 'World'
str_pad($str,$l,$p)	Pad string to a certain length	str_pad('5',3,'0',STR_PAD_LEFT) → '005'
str_repeat($str,$n)	Repeat a string n times	str_repeat('*',10)
str_split($str,$len)	Split string into array of chunks	str_split('Hello',2) → ['He','ll','o']
explode($del,$str)	Split string by delimiter into array	explode(',',$csv) → ['a','b','c']
implode($del,$arr)	Join array elements with delimiter	implode(', ',$names) → 'Alice, Bob'
sprintf($fmt,...$v)	Return formatted string	sprintf('%.2f', 3.14159) → '3.14'
number_format($n,$d)	Format with thousand separators	number_format(1234567,2) → '1,234,567.00'
htmlspecialchars($str)	Convert special chars to HTML entities	htmlspecialchars('<script>') → '<script>'
strip_tags($str)	Remove all HTML tags from string	strip_tags('Hello') → 'Hello'
md5($str)	Return 32-char MD5 hash	md5('password')

Function	Purpose	Example
hash($algo,$str)	Hash with specified algorithm	hash('sha256','password')
password_hash($str,$a)	Secure password hashing (bcrypt)	password_hash($pwd,PASSWORD_BCRYPT)
password_verify($p,$h)	Verify password against hash	password_verify($input,$hash)
preg_match($pat,$str)	Check if string matches regex pattern	preg_match('/^[a-z]+$/',$str)
preg_replace($p,$r,$s)	Replace matches of regex pattern	preg_replace('/\s+/',' ',$str)

Chapter 8 — Forms: GET, POST, Validation & Sanitization

Forms are the main way users send data to a PHP application. Understanding how to receive, validate, and sanitize form data is one of the most critical PHP skills.

8.1 HTML Form Sending Data to PHP

```
<!-- contact.html -->
<form action="process.php" method="POST">
  <label for="name">Full Name:</label>
  <input type="text" id="name" name="name" required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>

  <label for="message">Message:</label>
  <textarea id="message" name="message" rows="5"></textarea>

  <label for="subject">Subject:</label>
  <select id="subject" name="subject">
    <option value="general">General Enquiry</option>
    <option value="courses">About Courses</option>
    <option value="support">Technical Support</option>
  </select>

  <button type="submit">Send Message</button>
</form>
```

8.2 Processing the Form in PHP

```
<?php
// process.php

// Only process if form was submitted via POST
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // 1. Retrieve raw input
    $rawName    = $_POST['name']    ?? '';
    $rawEmail   = $_POST['email']   ?? '';
    $rawMessage = $_POST['message'] ?? '';
    $subject    = $_POST['subject'] ?? 'general';

    // 2. Sanitize — clean the data
    $name    = htmlspecialchars(trim($rawName));
```

```
    $email   = filter_var(trim($rawEmail), FILTER_SANITIZE_EMAIL);
    $message = htmlspecialchars(trim($rawMessage));

    // 3. Validate — check the data is correct
    $errors = [];

    if (empty($name)) {
        $errors[] = 'Name is required.';
    } elseif (strlen($name) < 2) {
        $errors[] = 'Name must be at least 2 characters.';
    }

    if (empty($email)) {
        $errors[] = 'Email is required.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors[] = 'Please enter a valid email address.';
    }

    if (empty($message)) {
        $errors[] = 'Message is required.';
    } elseif (strlen($message) < 10) {
        $errors[] = 'Message must be at least 10 characters.';
    }

    // 4. Handle results
    if (!empty($errors)) {
        foreach ($errors as $error) {
            echo "<p style='color:red'>Error: {$error}</p>";
        }
    } else {
        // All good — save to DB, send email, etc.
        echo "<p>Thank you, {$name}! Your message has been received.</p>";
    }

} else {
    header('Location: contact.html');
    exit();
}
?>
```

8.3 PHP Filter Functions

Filter Constant	Purpose	Common Use
FILTER_SANITIZE_STRING	Remove HTML tags (deprecated in PHP 8.1)	Sanitize text input
FILTER_SANITIZE_EMAIL	Remove illegal characters from email string	Clean email before validate
FILTER_SANITIZE_URL	Remove illegal characters from URL	Clean URL input
FILTER_SANITIZE_NUMBER_INT	Remove non-numeric chars (keeps - +)	Clean numeric input
FILTER_VALIDATE_EMAIL	Validate email format — returns email or false	Check email is valid format
FILTER_VALIDATE_URL	Validate URL — returns URL or false	Check if URL is valid
FILTER_VALIDATE_INT	Validate integer — returns int or false	Validate ID field
FILTER_VALIDATE_FLOAT	Validate float — returns float or false	Validate price field
FILTER_VALIDATE_IP	Validate IP address	Security logging
FILTER_VALIDATE_BOOLEAN	Validate boolean ('true','false','1','0','yes')	Validate toggle fields

SANITIZE FIRST — VALIDATE SECOND — ALWAYS

Sanitization: Clean the input — remove dangerous characters, encode HTML entities.
Validation: Check that the cleaned data is correct and expected.
NEVER store raw $_POST or $_GET data directly in a database.
NEVER output raw user input directly to the browser — use htmlspecialchars() always.
Always use htmlspecialchars() before echoing any user-supplied data to prevent XSS.

Chapter 9 — Superglobals

Superglobals are built-in PHP arrays that are always available in every scope — inside functions, classes, or anywhere in your script. They carry information about the HTTP request, server, session, cookies, files, and more.

9.1 $_GET — URL Query Parameters

```
<?php
// URL: http://mysite.co.ke/search.php?q=PHP&page=2

$query = isset($_GET['q'])    ? htmlspecialchars($_GET['q'])    : '';
$page  = isset($_GET['page']) ? (int)$_GET['page']              : 1;

echo "Searching for: {$query} — Page {$page}";
// Output: Searching for: PHP — Page 2

// NEVER use $_GET data directly in SQL — always sanitize/validate first
?>
```

9.2 $_POST — Form Submission Data

```
<?php
// Safely read a POST value with a default fallback
$email    = isset($_POST['email'])    ? $_POST['email']    : '';
$password = isset($_POST['password']) ? $_POST['password'] : '';

// PHP 7+ shorthand with null coalescing
$email    = $_POST['email']    ?? '';
$password = $_POST['password'] ?? '';

// Check if form was submitted
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Process the form
}
?>
```

9.3 $_SERVER — Server & Request Information

```
<?php
echo $_SERVER['PHP_SELF'];       // Current script: /form.php
echo $_SERVER['REQUEST_METHOD']; // GET or POST
echo $_SERVER['HTTP_HOST'];      // mysite.co.ke
echo $_SERVER['DOCUMENT_ROOT'];  // /var/www/html
echo $_SERVER['REMOTE_ADDR'];    // User's IP address
```

```
echo $_SERVER['HTTP_USER_AGENT'];// Browser info
echo $_SERVER['SERVER_NAME'];    // Server hostname
echo $_SERVER['QUERY_STRING'];   // ?q=php&page=2
echo $_SERVER['REQUEST_URI'];    // /search.php?q=php
echo $_SERVER['HTTPS'];          // 'on' if HTTPS
?>
```

9.4 $_SESSION — User Sessions

```
<?php
// MUST call session_start() at the very top of EVERY page that uses sessions
session_start();

// Store data in session
$_SESSION['user_id']   = 42;
$_SESSION['username']  = 'lucky_nakola';
$_SESSION['role']      = 'admin';
$_SESSION['logged_in'] = true;

// Read session data
if (isset($_SESSION['logged_in']) && $_SESSION['logged_in'] === true) {
    echo "Welcome back, {$_SESSION['username']}!";
} else {
    header('Location: login.php');
    exit();
}

// Remove a single session variable
unset($_SESSION['temp_data']);

// Destroy the entire session (logout)
session_unset();
session_destroy();
header('Location: login.php');
exit();
?>
```

9.5 $_COOKIE — Browser Cookies

```
<?php
// Set a cookie (must be done BEFORE any HTML output)
// setcookie(name, value, expiry, path, domain, secure, httponly)
setcookie('theme',        'dark',         time() + (86400 * 30), '/', '', false,
true);
setcookie('remember_me',  'lucky@ex.com', time() + (86400 * 30), '/', '', true,
true);
//                                        30 days from now       root  https
httponly

// Read a cookie
```

```
$theme = $_COOKIE['theme'] ?? 'light';
echo "Current theme: {$theme}";

// Delete a cookie — set expiry in the past
setcookie('theme', '', time() - 3600, '/');
?>
```

9.6 $_FILES — File Uploads

```
<!-- HTML form — enctype is REQUIRED for file uploads -->
<form action="upload.php" method="POST" enctype="multipart/form-data">
  <input type="file" name="profile_photo" accept="image/*">
  <button type="submit">Upload</button>
</form>

<?php
// upload.php
if (isset($_FILES['profile_photo'])) {
    $file = $_FILES['profile_photo'];

    // $_FILES contains: name, type, tmp_name, error, size
    echo $file['name'];      // original filename: photo.jpg
    echo $file['type'];      // MIME type: image/jpeg
    echo $file['size'];      // size in bytes: 102400
    echo $file['tmp_name'];  // temp path on server: /tmp/php7h8h9j
    echo $file['error'];     // 0 = no error

    // Validate
    $allowedTypes = ['image/jpeg', 'image/png', 'image/webp'];
    $maxSize      = 2 * 1024 * 1024;  // 2MB

    if (!in_array($file['type'], $allowedTypes)) {
        die('Only JPEG, PNG, WebP allowed.');
    }
    if ($file['size'] > $maxSize) {
        die('File too large. Maximum 2MB.');
    }
    if ($file['error'] !== UPLOAD_ERR_OK) {
        die('Upload error: ' . $file['error']);
    }

    // Create a safe unique filename
    $ext      = pathinfo($file['name'], PATHINFO_EXTENSION);
    $newName  = uniqid('photo_', true) . '.' . $ext;
    $uploadDir = 'uploads/';
    $dest     = $uploadDir . $newName;

    // Move from temp location to final destination
    if (move_uploaded_file($file['tmp_name'], $dest)) {
        echo "File uploaded successfully: {$dest}";
```

```
    } else {
        echo 'Upload failed.';
    }
}
?>
```

Chapter 10 — File Handling: Read, Write & Manage

10.1 Reading Files

```
<?php
// Read entire file into a string
$content = file_get_contents('data/students.txt');
echo $content;

// Read file into an array (one element per line)
$lines = file('data/names.txt', FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
foreach ($lines as $line) {
    echo htmlspecialchars($line) . '<br>';
}

// Read with fopen (more control)
$handle = fopen('data/log.txt', 'r');
if ($handle) {
    while (($line = fgets($handle)) !== false) {
        echo htmlspecialchars($line) . '<br>';
    }
    fclose($handle);
}
?>
```

10.2 Writing Files

```
<?php
// Write (overwrites if file exists, creates if not)
file_put_contents('data/output.txt', 'Hello from PHP!');

// Append to file
$log = date('Y-m-d H:i:s') . ' — User logged in\n';
file_put_contents('logs/app.log', $log, FILE_APPEND | LOCK_EX);

// fopen with mode 'w' (write), 'a' (append), 'r+' (read+write)
$handle = fopen('data/report.txt', 'w');
if ($handle) {
    fwrite($handle, "Student Report\n");
    fwrite($handle, "Generated: " . date('Y-m-d') . "\n");
    fclose($handle);
}
?>
```

10.3 File & Directory Functions

Function	Purpose
file_exists($path)	Check if a file or directory exists — returns bool.
is_file($path)	Check if path is a regular file.
is_dir($path)	Check if path is a directory.
is_readable($path)	Check if file can be read.
is_writable($path)	Check if file can be written to.
filesize($path)	Get file size in bytes.
filetype($path)	Get file type: file, dir, link.
copy($src,$dest)	Copy a file from source to destination.
rename($old,$new)	Rename or move a file.
unlink($path)	Delete a file permanently.
mkdir($path,0755,true)	Create directory (and parent dirs with true).
rmdir($path)	Remove an empty directory.
scandir($path)	List files and directories in a path.
glob($pattern)	Find files matching a pattern: glob('logs/*.log')
pathinfo($path)	Returns array: dirname, basename, extension, filename.
basename($path)	Get filename from a full path.
dirname($path)	Get directory from a full path.
realpath($path)	Get the absolute path, resolving symlinks.

Chapter 11 — Object-Oriented PHP: Classes, Objects & Inheritance

Object-Oriented Programming (OOP) organises code into classes — blueprints for creating objects. Each object has properties (data) and methods (functions). OOP makes large PHP applications organised, reusable, and maintainable.

11.1 Classes and Objects

```
<?php
class Student {

    // Properties (instance variables)
    public string $name;
    public string $email;
    private float $balance = 0.0;
    protected array $courses = [];
    public static int $totalStudents = 0;

    // Constructor — runs when object is created
    public function __construct(string $name, string $email) {
        $this->name  = $name;
        $this->email = $email;
        self::$totalStudents++;
    }

    // Public method
    public function enroll(string $course): void {
        $this->courses[] = $course;
        echo "{$this->name} enrolled in {$course}.\n";
    }

    // Getter for private property
    public function getBalance(): float {
        return $this->balance;
    }

    // Setter with validation
    public function setBalance(float $amount): void {
        if ($amount < 0) throw new InvalidArgumentException('Balance cannot be
negative.');
        $this->balance = $amount;
    }

    // Static method
    public static function getTotal(): int {
```

```
        return self::$totalStudents;
    }

    // Magic method — called when object used as string
    public function __toString(): string {
        return "{$this->name} ({$this->email})";
    }
    // Destructor — runs when object is destroyed
    public function __destruct() {
        // Cleanup code here
    }
}

// Create objects
$s1 = new Student('Lucky Nakola', 'lucky@example.com');
$s2 = new Student('Alice Wanjiru', 'alice@example.com');

$s1->enroll('PHP Development');
$s1->enroll('MySQL Foundations');
$s1->setBalance(15000.00);

echo $s1;                          // Lucky Nakola (lucky@example.com)
echo Student::getTotal();          // 2
echo $s1->getBalance();            // 15000
?>
```

11.2 Access Modifiers

Modifier	Scope & Access
public	Accessible from ANYWHERE — inside the class, outside, and in subclasses.
protected	Accessible inside the class and in SUBCLASSES only. Not from outside.
private	Accessible ONLY inside the class where it is defined. Not even subclasses.
static	Belongs to the CLASS itself, not to any instance. Access with ClassName::$property.
readonly	PHP 8.1+ — can only be set once (in constructor). Cannot be changed after.

11.3 Inheritance

```php
<?php
class Person {
    public function __construct(
        protected string $name,
        protected string $email
    ) {}

    public function greet(): string {
        return "Hello, I am {$this->name}.";
    }
}

// Student EXTENDS Person — inherits all its properties and methods
class Student extends Person {
    private array $courses = [];

    public function __construct(string $name, string $email, private string
$studentId) {
        parent::__construct($name, $email);  // Call parent constructor
    }

    // Override the greet method
    public function greet(): string {
        return parent::greet() . " I am a student (ID: {$this->studentId}).";
    }

    public function enroll(string $course): void {
        $this->courses[] = $course;
    }
}

class Instructor extends Person {
    public function __construct(string $name, string $email, private string
$subject) {
        parent::__construct($name, $email);
    }

    public function teach(): string {
        return "{$this->name} is teaching {$this->subject}.";
    }
}

$student    = new Student('Alice', 'alice@example.com', 'STU001');
$instructor = new Instructor('Lucky', 'lucky@example.com', 'PHP');

echo $student->greet();     // Hello, I am Alice. I am a student (ID: STU001).
echo $instructor->teach();  // Lucky is teaching PHP.
?>
```

11.4 Interfaces & Abstract Classes

```php
<?php
// Interface — defines a CONTRACT. Classes implementing it MUST have these
methods.
interface Payable {
    public function processPayment(float $amount): bool;
    public function getTransactionId(): string;
}

// Abstract class — cannot be instantiated directly. Forces subclasses to
implement.
abstract class PaymentGateway {
    abstract public function connect(): void;

    public function formatAmount(float $amount): string {
        return 'KES ' . number_format($amount, 2);
    }
}

// Concrete class implementing both
class MpesaGateway extends PaymentGateway implements Payable {
    private string $transactionId = '';

    public function connect(): void {
        echo 'Connecting to Mpesa API...';
    }

    public function processPayment(float $amount): bool {
        $this->transactionId = 'MPE' . rand(100000, 999999);
        echo 'Paid: ' . $this->formatAmount($amount);
        return true;
    }

    public function getTransactionId(): string {
        return $this->transactionId;
    }
}

$mpesa = new MpesaGateway();
$mpesa->connect();
$mpesa->processPayment(1500.00);
echo $mpesa->getTransactionId();
?>
```

11.5 Traits

```php
<?php
// Traits — reusable code blocks mixed into classes (PHP has no multiple
inheritance)
trait Timestamps {
    private string $createdAt;
    private string $updatedAt;

    public function setCreatedAt(): void {
        $this->createdAt = date('Y-m-d H:i:s');
    }
    public function getCreatedAt(): string { return $this->createdAt; }
}

trait SoftDelete {
    private bool $isDeleted = false;
    private ?string $deletedAt = null;

    public function softDelete(): void {
        $this->isDeleted  = true;
        $this->deletedAt  = date('Y-m-d H:i:s');
    }
    public function isDeleted(): bool { return $this->isDeleted; }
}

class Post {
    use Timestamps, SoftDelete;  // Use BOTH traits

    public function __construct(public string $title) {
        $this->setCreatedAt();
    }
}

$post = new Post('My First PHP Post');
echo $post->getCreatedAt();  // 2025-06-01 10:30:00
$post->softDelete();
echo $post->isDeleted() ? 'Deleted' : 'Active';  // Deleted
?>
```

Chapter 12 — PHP & MySQL: CRUD with MySQLi & PDO

Connecting PHP to MySQL is the foundation of every dynamic website. PHP can create, read, update, and delete database records — making it possible to build login systems, blog platforms, e-commerce stores, school portals, and dashboards.

12.1 Connecting to MySQL

```
<?php
// === config.php — Database configuration ===
define('DB_HOST', 'localhost');
define('DB_USER', 'root');
define('DB_PASS', '');          // Your MySQL password
define('DB_NAME', 'school_db');

// METHOD 1: MySQLi (procedural) — simpler for beginners
$conn = mysqli_connect(DB_HOST, DB_USER, DB_PASS, DB_NAME);

if (!$conn) {
    die('Connection failed: ' . mysqli_connect_error());
}
echo 'Connected successfully!';

// METHOD 2: PDO — more powerful, supports many databases
try {
    $pdo = new PDO(
        'mysql:host=' . DB_HOST . ';dbname=' . DB_NAME . ';charset=utf8mb4',
        DB_USER,
        DB_PASS,
        [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
         PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]
    );
    echo 'PDO Connected!';
} catch (PDOException $e) {
    die('PDO Connection failed: ' . $e->getMessage());
}
?>
```

12.2 CREATE — Insert Records

```php
<?php
require 'config.php';

// ❌ DANGEROUS — vulnerable to SQL injection
// $sql = "INSERT INTO students (name, email) VALUES ('$name', '$email')";

// ☑ SAFE — MySQLi Prepared Statements
$name  = 'Lucky Nakola';
$email = 'lucky@example.com';
$course= 'PHP Development';

$stmt = mysqli_prepare($conn, 'INSERT INTO students (name, email, course) VALUES
(?, ?, ?)');
mysqli_stmt_bind_param($stmt, 'sss', $name, $email, $course);
// 'sss' = three strings. Use 'i'=integer, 'd'=double, 'b'=blob

if (mysqli_stmt_execute($stmt)) {
    $newId = mysqli_insert_id($conn);
    echo "Student created with ID: {$newId}";
} else {
    echo 'Error: ' . mysqli_stmt_error($stmt);
}
mysqli_stmt_close($stmt);

// ☑ SAFE — PDO Prepared Statement
$sql  = 'INSERT INTO students (name, email, course) VALUES (:name, :email,
:course)';
$stmt = $pdo->prepare($sql);
$stmt->execute([':name'=>$name, ':email'=>$email, ':course'=>$course]);
echo 'New ID: ' . $pdo->lastInsertId();
?>
```

12.3 READ — Fetch Records

```php
<?php
// Fetch ALL students
$result = mysqli_query($conn, 'SELECT * FROM students ORDER BY name ASC');

while ($row = mysqli_fetch_assoc($result)) {
    echo "{$row['id']}. {$row['name']} — {$row['course']}\n";
}
mysqli_free_result($result);

// Fetch ONE student by ID — prepared statement
$id   = 5;
$stmt = mysqli_prepare($conn, 'SELECT * FROM students WHERE id = ?');
mysqli_stmt_bind_param($stmt, 'i', $id);
mysqli_stmt_execute($stmt);
```

```
$result = mysqli_stmt_get_result($stmt);
$student = mysqli_fetch_assoc($result);

if ($student) {
    echo "Found: {$student['name']}";
} else {
    echo 'Student not found.';
}

// PDO — fetch all into an array
$stmt = $pdo->prepare('SELECT * FROM students WHERE course = :course');
$stmt->execute([':course' => 'PHP Development']);
$students = $stmt->fetchAll();   // Array of all rows

foreach ($students as $s) {
    echo "{$s['name']}\n";
}

// PDO — fetch single row
$stmt = $pdo->prepare('SELECT * FROM students WHERE id = :id');
$stmt->execute([':id' => 1]);
$student = $stmt->fetch();   // One row as assoc array
?>
```

12.4 UPDATE & DELETE

```
<?php
// UPDATE — change email for student ID 5
$newEmail = 'new@example.com';
$id       = 5;

$stmt = mysqli_prepare($conn, 'UPDATE students SET email = ? WHERE id = ?');
mysqli_stmt_bind_param($stmt, 'si', $newEmail, $id);
mysqli_stmt_execute($stmt);

echo 'Rows updated: ' . mysqli_stmt_affected_rows($stmt);

// PDO UPDATE
$stmt = $pdo->prepare('UPDATE students SET email = :email WHERE id = :id');
$stmt->execute([':email' => $newEmail, ':id' => $id]);
echo 'Updated: ' . $stmt->rowCount() . ' row(s)';

// DELETE — remove student
$stmt = $pdo->prepare('DELETE FROM students WHERE id = :id');
$stmt->execute([':id' => 10]);
echo 'Deleted: ' . $stmt->rowCount() . ' row(s)';

// Close connection
mysqli_close($conn);  // MySQLi
```

```
$pdo = null;          // PDO
?>
```

ALWAYS USE PREPARED STATEMENTS — SQL INJECTION IS THE #1 WEB ATTACK

SQL Injection happens when user input is inserted directly into SQL queries.
A hacker can type: ' OR '1'='1 into a login form and bypass authentication.
Prepared statements separate SQL logic from data — the data is NEVER executed as SQL.
Rule: NEVER build SQL queries with string concatenation using user input.
Always use: mysqli_prepare() + bind_param() OR PDO prepare() + execute()

Chapter 13 — Sessions, Cookies & Authentication Systems

Authentication is the process of verifying a user's identity — allowing them to log in, stay logged in across pages, and log out securely. PHP sessions and cookies are the tools that make this possible.

13.1 Complete Login System

```
<?php
// login.php
session_start();
require 'config.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $email    = filter_var(trim($_POST['email'] ?? ''), FILTER_SANITIZE_EMAIL);
    $password = trim($_POST['password'] ?? '');

    // Basic validation
    if (empty($email) || empty($password)) {
        $error = 'Both fields are required.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = 'Invalid email format.';
    } else {
        // Find user by email (prepared statement)
        $stmt = $pdo->prepare('SELECT id, name, email, password_hash, role FROM
users WHERE email = :email LIMIT 1');
        $stmt->execute([':email' => $email]);
        $user = $stmt->fetch();

        if ($user && password_verify($password, $user['password_hash'])) {
            // ☑ Credentials match — create session
            session_regenerate_id(true);  // Prevent session fixation attack
            $_SESSION['user_id']   = $user['id'];
            $_SESSION['username']  = $user['name'];
            $_SESSION['role']      = $user['role'];
            $_SESSION['logged_in'] = true;

            header('Location: dashboard.php');
            exit();
        } else {
            $error = 'Invalid credentials.';  // Vague on purpose — security
        }
    }
}
?>
```

13.2 Registration — Storing Passwords Securely

```
<?php
// register.php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name     = htmlspecialchars(trim($_POST['name']     ?? ''));
    $email    = filter_var(trim($_POST['email']    ?? ''), FILTER_SANITIZE_EMAIL);
    $password = $_POST['password'] ?? '';
    $confirm  = $_POST['confirm']  ?? '';

    $errors = [];
    if (empty($name))                                   $errors[] = 'Name
required.';
    if (!filter_var($email, FILTER_VALIDATE_EMAIL))     $errors[] = 'Valid email
required.';
    if (strlen($password) < 8)                          $errors[] = 'Password min
8 chars.';
    if ($password !== $confirm)                         $errors[] = 'Passwords do
not match.';

    // Check if email already registered
    $stmt = $pdo->prepare('SELECT id FROM users WHERE email = :email');
    $stmt->execute([':email' => $email]);
    if ($stmt->fetch()) $errors[] = 'Email already registered.';

    if (empty($errors)) {
        // NEVER store plain text passwords — hash with bcrypt
        $hash = password_hash($password, PASSWORD_BCRYPT);

        $stmt = $pdo->prepare(
            'INSERT INTO users (name, email, password_hash) VALUES (:name, :email,
:hash)'
        );
        $stmt->execute([':name'=>$name, ':email'=>$email, ':hash'=>$hash]);

        header('Location: login.php?registered=1');
        exit();
    }
}
?>
```

13.3 Protecting Pages — Auth Guard

```
<?php
// auth.php — include at the top of every protected page
session_start();

function requireLogin(string $redirect = 'login.php'): void {
    if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !== true) {
```

```
        header('Location: ' . $redirect);
        exit();
    }
}

function requireRole(string $role): void {
    requireLogin();
    if ($_SESSION['role'] !== $role) {
        header('Location: unauthorized.php');
        exit();
    }
}

// Usage in any protected file:
require 'auth.php';
requireLogin();         // Any logged-in user can access
requireRole('admin');   // Only admin can access
?>
```

Chapter 14 — Error Handling, Debugging & Security

14.1 PHP Error Levels

Error Level	Description
E_ERROR	Fatal error — script stops immediately. Cannot be handled.
E_WARNING	Non-fatal warning — script continues. Should be fixed.
E_NOTICE	Minor notice — often undefined variables. Best practice to fix.
E_PARSE	Parse/syntax error — script cannot run at all.
E_DEPRECATED	Function/feature will be removed in future PHP versions.
E_ALL	All errors, warnings, and notices — use during development.

14.2 Error Handling with try/catch

```
<?php
function divideNumbers(int $a, int $b): float {
    if ($b === 0) {
        throw new InvalidArgumentException('Division by zero is not allowed.');
    }
    return $a / $b;
}

try {
    echo divideNumbers(10, 2);    // 5
    echo divideNumbers(10, 0);    // Throws exception
} catch (InvalidArgumentException $e) {
    echo 'Caught: ' . $e->getMessage();
} catch (Exception $e) {
    echo 'General error: ' . $e->getMessage();
} finally {
    echo 'This always runs.';
}

// Multiple exception types (PHP 8)
try {
    // risky operation
} catch (TypeError | ValueError $e) {
    echo 'Type or Value error: ' . $e->getMessage();
}
?>
```

14.3 Custom Exception Classes

```php
<?php
class DatabaseException extends RuntimeException {}
class ValidationException extends InvalidArgumentException {
    private array $errors;
    public function __construct(array $errors) {
        $this->errors = $errors;
        parent::__construct('Validation failed.');
    }
    public function getErrors(): array { return $this->errors; }
}

try {
    throw new ValidationException(['Name is required', 'Email is invalid']);
} catch (ValidationException $e) {
    foreach ($e->getErrors() as $error) {
        echo "- {$error}\n";
    }
}
?>
```

14.4 PHP Security Best Practices

Vulnerability / Practice	How to Handle It Securely
SQL Injection	ALWAYS use prepared statements. NEVER put user input directly in SQL queries.
XSS Prevention	ALWAYS use htmlspecialchars() before outputting any user data to HTML.
CSRF Protection	Use CSRF tokens in all forms — unique per-session hidden field.
Password Security	ALWAYS hash with password_hash(PASSWORD_BCRYPT). Never MD5 or SHA1 for passwords.
Session Security	Use session_regenerate_id(true) after login. Set session cookie to HttpOnly.
Input Validation	Validate ALL inputs server-side. Never trust client-side validation alone.
File Uploads	Validate MIME type, extension, and size. Use uniqid() to rename. Store outside webroot.
Error Display	In production: display_errors=Off, log_errors=On. Never show errors to users.
HTTPS	Always use HTTPS in production. Set Secure flag on cookies.
Directory Security	Add .htaccess to restrict direct access to sensitive directories.
include/require	Never use user input in file include paths. Leads to LFI (Local File Inclusion).

Vulnerability / Practice	How to Handle It Securely
eval()	Never use eval() with user input. Extremely dangerous.

Chapter 15 — PHP Best Practices, MVC & Project Structure

15.1 PHP Best Practices

Best Practice	How to Apply It
Use strict types	Add declare(strict_types=1); at the top — catches type errors.
Always use prepare()	For every query that involves variables — prevents SQL injection.
Separate config	Store DB credentials in a config.php file outside the webroot.
Use require_once	For class files, config, and helpers — prevents double-include errors.
PSR coding standards	Follow PHP-FIG standards: PSR-1 (basic), PSR-12 (coding style).
Meaningful names	$userEmailAddress not $uea. getStudentById() not getData().
Single responsibility	Each function/class should do one thing well.
Document with PHPDoc	Add /** @param string $name */ doc comments for IDEs and teams.
Use composer	PHP's dependency manager — install packages with: composer require
Log errors properly	Use error_log() or a logging library (Monolog). Never echo errors in production.
Validate & sanitize always	Client-side is UX. Server-side is security. NEVER skip server validation.
Protect sensitive routes	Always check session/auth at the top of protected pages.

15.2 Professional Project File Structure

```
my-php-project/
├── public/              ← Document root — only this folder is web-accessible
│   ├── index.php        ← Front controller (entry point)
│   ├── css/
│   ├── js/
│   └── uploads/
│
├── app/                 ← Application code — NOT web-accessible
│   ├── config/
│   │   └── database.php ← DB credentials
│   ├── controllers/
│   │   ├── HomeController.php
│   │   ├── AuthController.php
│   │   └── StudentController.php
```

```
│   ├── models/
│   │   ├── User.php
│   │   └── Student.php
│   ├── views/              ← HTML templates
│   │   ├── layouts/
│   │   │   └── main.php
│   │   ├── home/
│   │   │   └── index.php
│   │   └── auth/
│   │       ├── login.php
│   │       └── register.php
│   └── helpers/
│       └── functions.php
│
├── vendor/                 ← Composer packages (auto-generated)
├── logs/                   ← Error logs (NOT web-accessible)
├── .htaccess               ← Redirect all requests to public/index.php
├── composer.json
└── .env                    ← Environment variables (NEVER commit this!)
```

15.3 MVC Pattern Explained

MVC — Model, View, Controller — is the architectural pattern used by every major PHP framework (Laravel, Symfony, CodeIgniter). Even without a framework, understanding MVC makes your projects organised and professional.

Layer	Responsibility	Examples
Model	Handles data — database queries, business logic, validation rules.	User.php, Student.php, Product.php
View	Handles presentation — HTML templates that display data. No logic.	login.php, dashboard.php, home.php
Controller	Handles requests — receives input, calls models, passes data to views.	AuthController, StudentController

15.4 Your PHP Learning Roadmap

Stage	Topics & Technologies
NOW — Foundations	Syntax, variables, arrays, loops, functions, forms, MySQL, sessions.
NEXT — OOP & Security	Classes, inheritance, interfaces, traits. Prepared statements. Auth systems.
THEN — Framework	Laravel — the most popular PHP framework. Clean MVC, Eloquent ORM, Artisan CLI.
THEN — APIs	Build REST APIs with PHP. Return JSON responses. Connect to JS frontends.

Stage	Topics & Technologies
THEN — WordPress	Themes, plugins, custom post types — powers 43% of all websites.
ADVANCED	Composer packages, Unit Testing (PHPUnit), Docker, CI/CD, Microservices.

Chapter 16 — Comprehensive Quiz & Coding Challenges

Answer every section independently before reviewing answers with the class.

Section A — Multiple Choice (25 questions — 1 mark each)

1. What does PHP stand for?

   a) Personal Home Page b) PHP: Hypertext Preprocessor c) Private Hosting Platform d) Programmatic HTML Pages

2. Which symbol is used to declare all PHP variables?

   a) @ b) # c) $ d) %

3. PHP is a _____ language — it runs on the _____.

   a) client-side / browser b) server-side / server c) client-side / server d) markup / browser

4. Which of the following correctly outputs text in PHP?

   a) print_text('Hello'); b) console.log('Hello'); c) echo 'Hello'; d) output('Hello');

5. What is the result of: var_dump(5 === '5'); in PHP?

   a) true b) false c) 1 d) NULL

6. Which operator is used to concatenate strings in PHP?

   a) + b) & c) . d) *

7. How do you create an associative array in PHP?

   a) $arr = [0=>'PHP', 1=>'MySQL']; b) $arr = ['language'=>'PHP', 'db'=>'MySQL'];
   c) $arr = array('PHP', 'MySQL'); d) $arr = {language:'PHP', db:'MySQL'};

8. Which loop always executes its body at least once?

   a) for b) while c) foreach d) do...while

9. What does the null coalescing operator (??) return?

   a) The left value if it is truthy, otherwise the right
   b) The left value if it is NOT null or undefined, otherwise the right
   c) Always returns null
   d) Checks both values for null and returns the one that is not

10. What is the correct way to start a PHP session?

a) session_begin(); b) start_session(); c) session_start(); d) $_SESSION = start();

11. Which function securely hashes a password in PHP?
    a) md5() b) sha1() c) base64_encode() d) password_hash()

12. Which PHP superglobal contains form data sent via the POST method?
    a) $_GET b) $_FORM c) $_POST d) $_REQUEST

13. What does htmlspecialchars() protect against?
    a) SQL Injection b) Cross-Site Scripting (XSS) c) CSRF attacks d) Brute force login

14. What is the purpose of a prepared statement in PHP?
    a) To make SQL run faster by caching results
    b) To prevent SQL injection by separating SQL logic from data
    c) To format database query results as HTML
    d) To automatically validate form inputs before database insertion

15. Which PHP function checks if a variable is set AND is not null?
    a) empty() b) defined() c) isset() d) exists()

16. What does the static keyword do inside a PHP function?
    a) Makes the variable accessible from outside the function
    b) Prevents the variable from being changed
    c) Makes the variable retain its value between function calls
    d) Makes the variable a constant

17. In OOP PHP, which keyword is used to call a parent class constructor?
    a) base() b) parent::__construct() c) super() d) extends()

18. What is the difference between require and include in PHP?
    a) They are identical — just different names
    b) require stops the script with a fatal error if file not found; include just warns
    c) include stops with a fatal error; require only warns
    d) require only works with .php files; include works with any file type

19. Which function removes whitespace from both ends of a string?
    a) strip() b) clean() c) trim() d) chop()

20. What does session_regenerate_id(true) do and when should you use it?
    a) Creates a new session file and clears all data — use on every page load
    b) Generates a new session ID to prevent session fixation — use after successful login
    c) Regenerates the session encryption key — use weekly
    d) Resets the session timer — use when the user is inactive

21. Which attribute is REQUIRED on an HTML form for file uploads to work?

    a) method="file" b) action="upload.php" c) enctype="multipart/form-data" d) type="upload"

22. What does PDO stand for?

    a) PHP Data Operations b) PHP Database Object c) PHP Data Objects d) Portable Database Output

23. What is the MVC architectural pattern?

    a) Model-View-Controller — separates data, presentation, and request handling
    b) Main-Variable-Class — a way to declare PHP variables
    c) Method-Value-Construct — a PHP OOP design pattern
    d) MySQL-Variables-Code — a database design pattern

24. What does the match expression in PHP 8 use for comparison?

    a) Loose equality == b) Strict equality === c) Type juggling d) Regular expressions

25. Which function is used to move an uploaded file from its temporary location to a permanent one?

    a) copy() b) rename() c) file_put_contents() d) move_uploaded_file()

Section B — Short Answer (10 questions — 3 marks each)

1. Explain the difference between == and === in PHP. Why is === almost always the safer choice? Give an example where == produces an unexpected result.

2. What is SQL Injection? Give a real-world example of how a hacker could exploit it. How do prepared statements prevent it? Write the difference between a vulnerable query and a safe one.

3. Explain the difference between $_SESSION and $_COOKIE. How does each store data? When would you use one over the other?

4. Describe the password storage lifecycle in a PHP registration and login system. Why must you NEVER store plain-text passwords? Which functions should you use and how do they work together?

5. What is the difference between include and require? What is the difference between include_once and require_once? When would you use each?

6. Explain the concept of Object-Oriented Programming in PHP. Define: class, object, property, method, constructor, and inheritance. Use a real-world example.

7. What is the difference between a PHP interface and an abstract class? When would you use each? Can a class implement multiple interfaces? Can it extend multiple abstract classes?

8. Describe the three layers of MVC (Model, View, Controller). What responsibility does each layer have? Why does separating these concerns make applications easier to maintain?

9. List FIVE important PHP security practices and explain WHY each one is important. Give a brief example of what could go wrong if you skip each practice.

10. What is a PHP Trait and how does it differ from inheritance? Write an example of a Trait called 'Auditable' that adds 'created_at' and 'updated_at' timestamp properties, and show a class using it.

Section C — Coding Challenges

Challenge 1 — Complete Registration & Login System:

Build a fully functional PHP + MySQL mini authentication system with these files: (a) register.php — HTML form + PHP processing: name, email, password, confirm password. Validate all fields. Hash the password with bcrypt. Store in a 'users' table. Redirect to login on success. (b) login.php — HTML form + PHP processing: email and password. Fetch user from DB. Verify password with password_verify(). On success: set session variables (user_id, username, role, logged_in). Regenerate session ID. Redirect to dashboard. (c) dashboard.php — Protected page. Include auth.php guard at the top. Display welcome message with the user's name. Show logout button. (d) logout.php — Destroy session. Redirect to login. Use prepared statements for ALL database queries. Use htmlspecialchars() on all output.

Challenge 2 — Student Management CRUD:

Build a PHP + MySQL student management system: (a) Create a 'students' table with: id, name, email, course, marks, created_at. (b) index.php — Fetch and display ALL students in an HTML table. Include Edit and Delete buttons for each row. Show total count and class average marks. (c) add.php — Form and processing to add a new student. Validate: name (required, min 2 chars), email (valid format, unique), course (required), marks (integer 0–100). (d) edit.php — Pre-populated form to edit a student by ID. Update on submit. (e) delete.php — Confirm and delete a student by ID. (f) Use PDO with prepared statements throughout. Sanitize all output with htmlspecialchars().

Challenge 3 — File Upload & Gallery:

Build a PHP image upload and gallery system: (a) upload.php — Form with a file input. PHP processing: validate MIME type (JPEG, PNG, WebP only), maximum 2MB, UPLOAD_ERR_OK. Rename with uniqid() to prevent filename collisions. Move to 'uploads/' directory. Save filename, original name, upload date, and file size to a 'gallery' MySQL table using PDO. (b) gallery.php — Fetch all images from the database. Display them in a CSS grid. Show original name and upload date below each image. Add a Delete button. (c) delete_image.php — Remove image file from disk (unlink) and delete record from database. Handle the case where the file no longer exists on disk gracefully.

Section A — Answer Key

Multiple Choice Answers

1. b) PHP: Hypertext Preprocessor 2. c) $ 3. b) server-side / server
4. c) echo 'Hello' 5. b) false (strict comparison of different types)
6. c) . (dot) 7. b) Associative array with named keys
8. d) do...while 9. b) Left if not null/undefined, else right
10. c) session_start() 11. d) password_hash()
12. c) $_POST 13. b) XSS (Cross-Site Scripting)
14. b) Separates SQL logic from data 15. c) isset()
16. c) Retains value between calls 17. b) parent::__construct()
18. b) require stops; include just warns
19. c) trim() 20. b) Prevents session fixation — use after login
21. c) enctype="multipart/form-data" 22. c) PHP Data Objects
23. a) Model-View-Controller 24. b) Strict equality ===
25. d) move_uploaded_file()

Build real projects. Break things. Fix them. That is how PHP developers grow.

Get Techy With Lucky | Tech Pulse Insider | Instructor: Lucky Nakola